The second and third arguments are credentials: username and password, respectively. One gotcha to watch out for is the `host=...` section: if this is set to `host=localhost` – and you have a *nix system – then PHP will try to connect to the database via a special file (called a Unix Domain Socket) and not a TCP port, the source of a number of questions on https://stackoverflow.com

Executing Statements

Now that we have a PDO connection to the DB, we can run some MySQL statements. Right now the `testdb` database is empty, so we're going to create a table, which is how MySQL stores data. Think of tables as being a bit like spreadsheets; they have columns and rows – the columns are the fields, and each row is a record:

```
$sql = <<< SQL
CREATE TABLE IF NOT EXISTS `test` (
    `id` int(10) NOT NULL AUTO_INCREMENT,
    `message` varchar(255) NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1
SQL;

$pdo->exec($sql);
```

This example uses a HEREDOC string (all explained in Chapter 8, "Strings") to build a MySQL statement using the `<<<` operator. The funny-looking `$pdo->exec()` is a bit like a function call, but the function belongs to an object. (We are jumping a little bit ahead of ourselves here; all these language features are explained in the relevant chapters, and I promise not to do this again! Probably.) The SQL itself is a `CREATE TABLE` statement, which defines the `test` table with two columns: `id` and `message`. We can check the result of this by accessing the running DB server (with, you guessed it, another docker command) and running an SQL query from the cli:

```
$ docker exec -it mariadb mariadb -uroot -proot123 testdb
...
MariaDB [testdb]> DESC test;
```